SE-314: Software Construction (BESE 14AB)

Lab 05: Test First Programming (Tweet Tweet)

Student Handout: step-by-step guide to the lab tasks

Before you start

This handout goes with the Lab 05 manual. Read it once completely before you start, and then follow it step by step in Eclipse. In this lab the order of the steps matters more than usual: you will write the tests first and the code after that. If you write the code first and then make tests that simply agree with your code, you have missed the point of the lab.

You will work on Problems 1 and 2 of Problem Set 1 (Tweet Tweet) of MIT 6.005. The project has a few classes for working with tweets. In Task 1 you test and implement two methods of `Extract.java`, and in Task 2 you test and implement three methods of `Filter.java`.

What you need:

- Eclipse IDE for Java Developers (JUnit 4 is already included in it).
- Java 11 or newer. The JDK that comes with Eclipse is fine.
- The starter project. The link in the manual is https://ocw.mit.edu/ans7870/6/6.005/s16/psets/ps1/. If you cannot get a working Eclipse project from there, use **Lab05_ps1_Starter_EclipseProject.zip** shared on LMS. It has the same classes, specs and starting tests as the MIT problem set.

Two differences from the MIT version you should know about. First, the original `Main.java` downloaded tweets from an MIT server that is not running anymore, so in the LMS project `Main.java` reads a small file of sample tweets (`tweets.txt`) instead. Second, the MIT folder also has `SocialNetwork.java` and its test for Problems 3 and 4. Those are not part of this lab, so they are left out.

The screenshots in this handout were taken in Eclipse 2026-09 on a Linux machine. The windows look a little different from Windows, but the menus and dialogs are the same.

Test-first programming in short

The manual gives four steps. Here is what each one means in practice:

Step	What you actually do
1. Study the spec	Read the Javadoc comment above the method: the parameters, what it must return, and what it leaves open. Do not look at how you would code it yet.
2. Write tests	Write a testing strategy (partitions of the input and output), then write JUnit tests that cover every part. Run them. They will fail, and that is expected.
3. Implement	Now write the method so that it follows the spec.

Step	What you actually do
4. Revise	Run the tests again. Fix the code (or a wrong test) until everything passes. If you notice a case you did not test, add a test for it.

A **partition** divides all possible inputs into groups that should behave in a similar way, for example "list has 1 tweet" and "list has more than 1 tweet". You pick at least one test from each group. Above each test write a short comment saying which parts it covers, so that the strategy and the tests can be checked against each other.

One more term from the hints: a spec is **underdetermined** when it allows more than one correct answer. For example, `getMentionedUsers` may return "Alyssa" or "alyssa" for the same user. Your tests must accept every answer the spec allows. If a test only passes for your own choice, it is testing a stronger spec than the one given, and it would fail on another correct implementation. The comment at the end of the test files warns about exactly this.

Task 1: Extracting data from tweets

Step 1: Import the project into Eclipse

Go to **File > Import...**, open the **General** folder, select **Existing Projects into Workspace** and press **Next**.

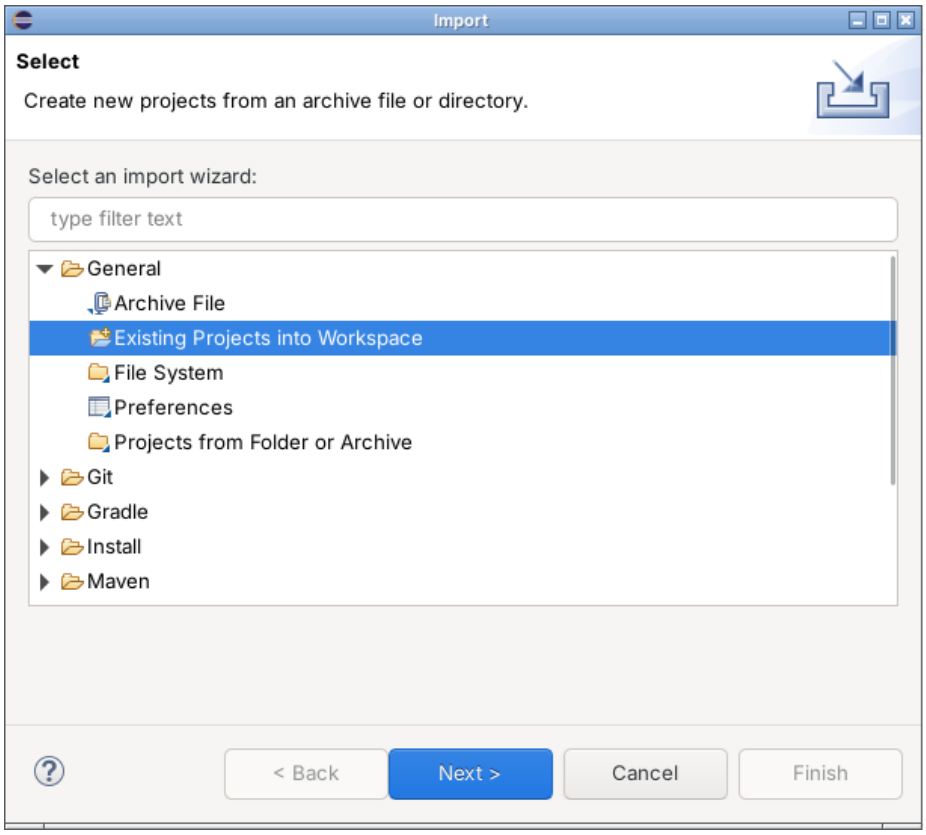


Figure 1: Existing Projects into Workspace

Choose **Select archive file**, browse to the downloaded zip and press **Finish**. (If you have already unzipped it, use Select root directory and point it to the ps1 folder instead.)

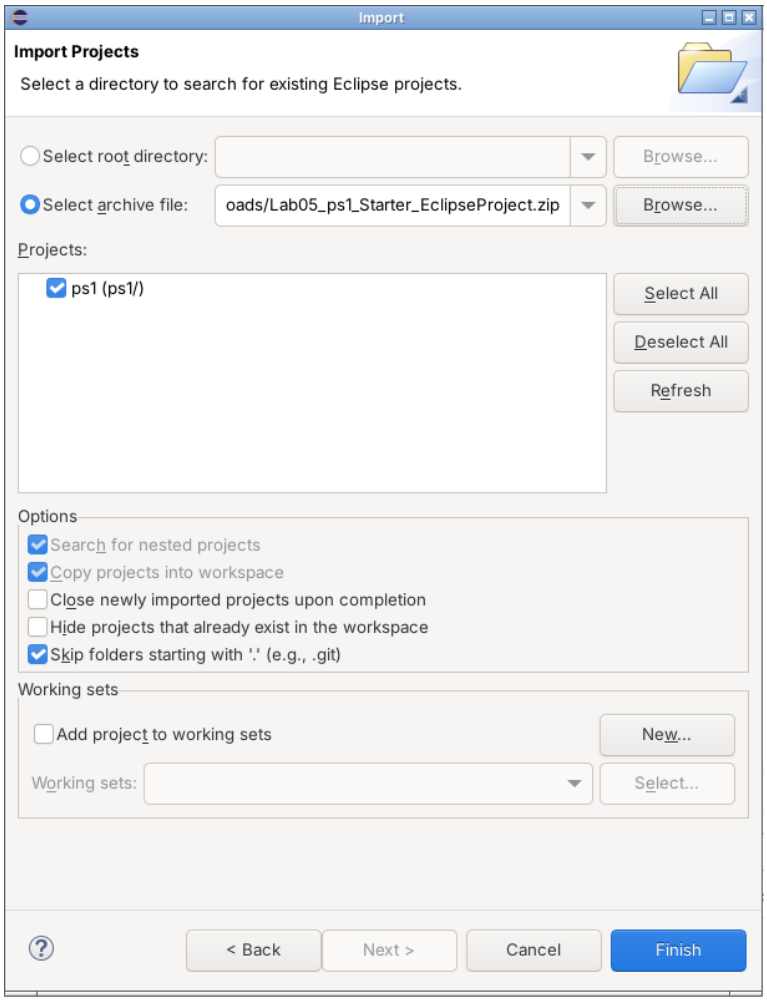


Figure 2: Importing the starter project from the zip file

Expand the project in the Package Explorer. The code is in `src/twitter` and the tests are in `test/twitter`. There should be no errors in the Problems view.

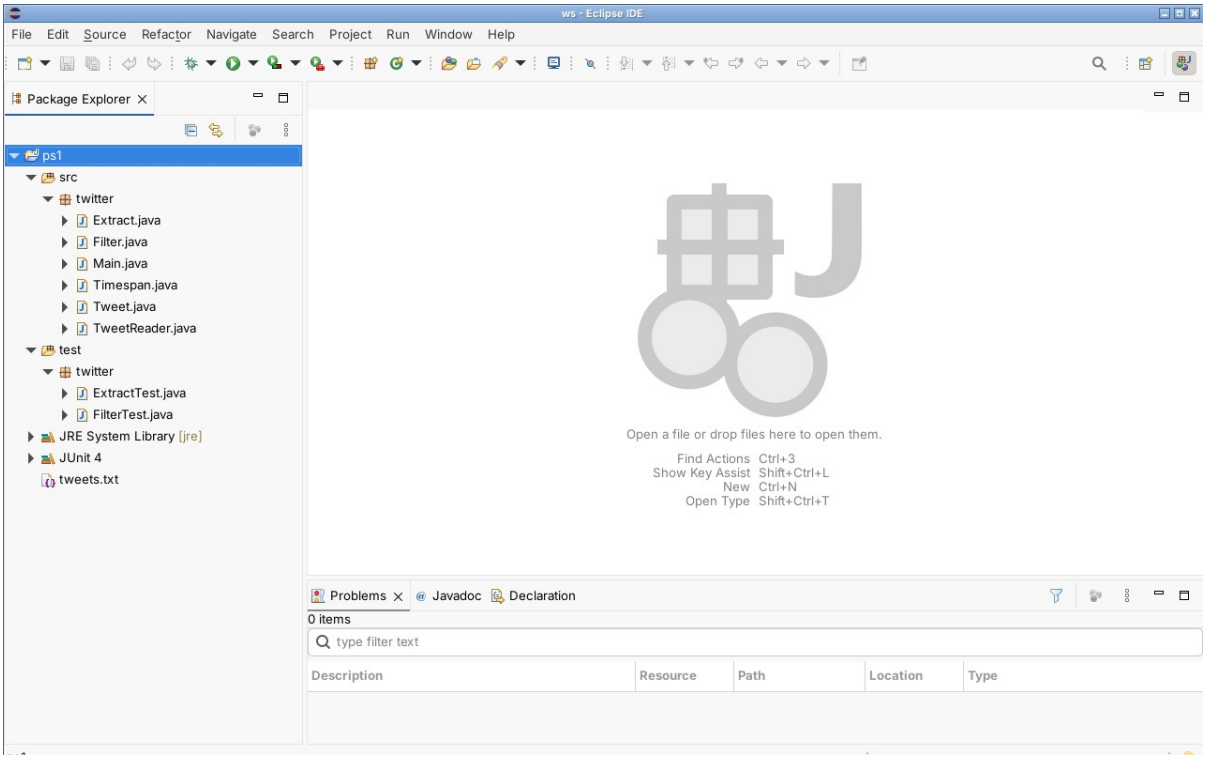


Figure 3: The ps1 project after import

If you see "org.junit cannot be resolved", right-click the project and choose **Build Path > Add Libraries... > JUnit > JUnit 4**.

Step 2: Read the specs

Open `Tweet.java` and `Timespan.java` first and read their comments. You will need these facts:

- A tweet has an id, an author, a text and a timestamp. The timestamp is a `java.time.Instant`. Two Instants are compared with `isBefore()` and `isAfter()`, not with < and >.
- A username is made of letters, digits, underscore (_) and hyphen (-), and usernames are case-insensitive. This is the spec of `Tweet.getAuthor()`.
- A `Timespan` has a start and an end, and it includes both endpoints. The start may be equal to the end.

Now open `Extract.java`. Both methods only throw "not implemented" at the moment. Read the two specs slowly, because all your tests will be based on them.

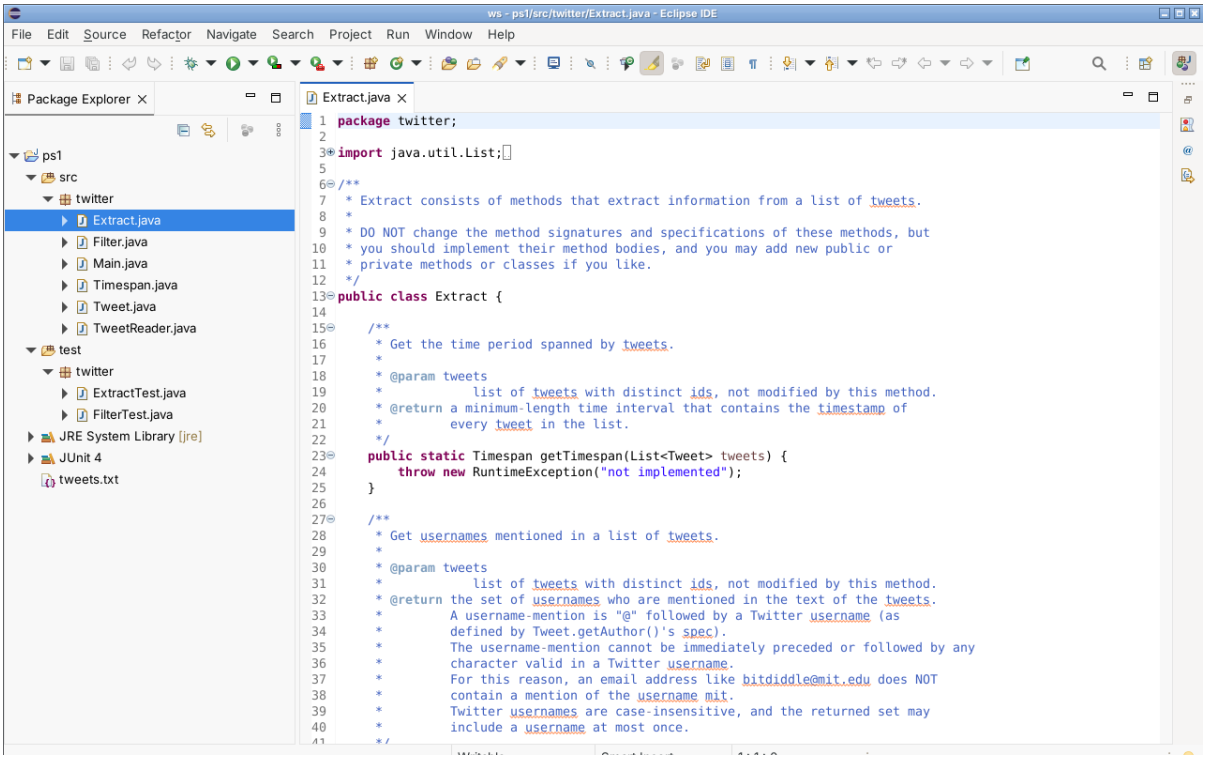


Figure 4: Specs of getTimespan and getMentionedUsers in Extract.java

Things to notice in these specs:

- `getTimespan` must return the **smallest** timespan that contains every tweet. So the start is the earliest timestamp and the end is the latest. The list is not necessarily in time order.
- The spec does not say what happens for an empty list. That case is underdetermined, so do not write a test for it.
- In `getMentionedUsers`, "@" must not come right after a username character. That is why bitdiddle@mit.edu does not mention "mit".
- The returned set can hold a username in any case, but each user only once.

Step 3: Run the given tests before writing any code

Open `ExtractTest.java`. It already has three tests: `testAssertionsEnabled`, `testGetTimespanTwoTweets` and `testGetMentionedUsersNoMention`.

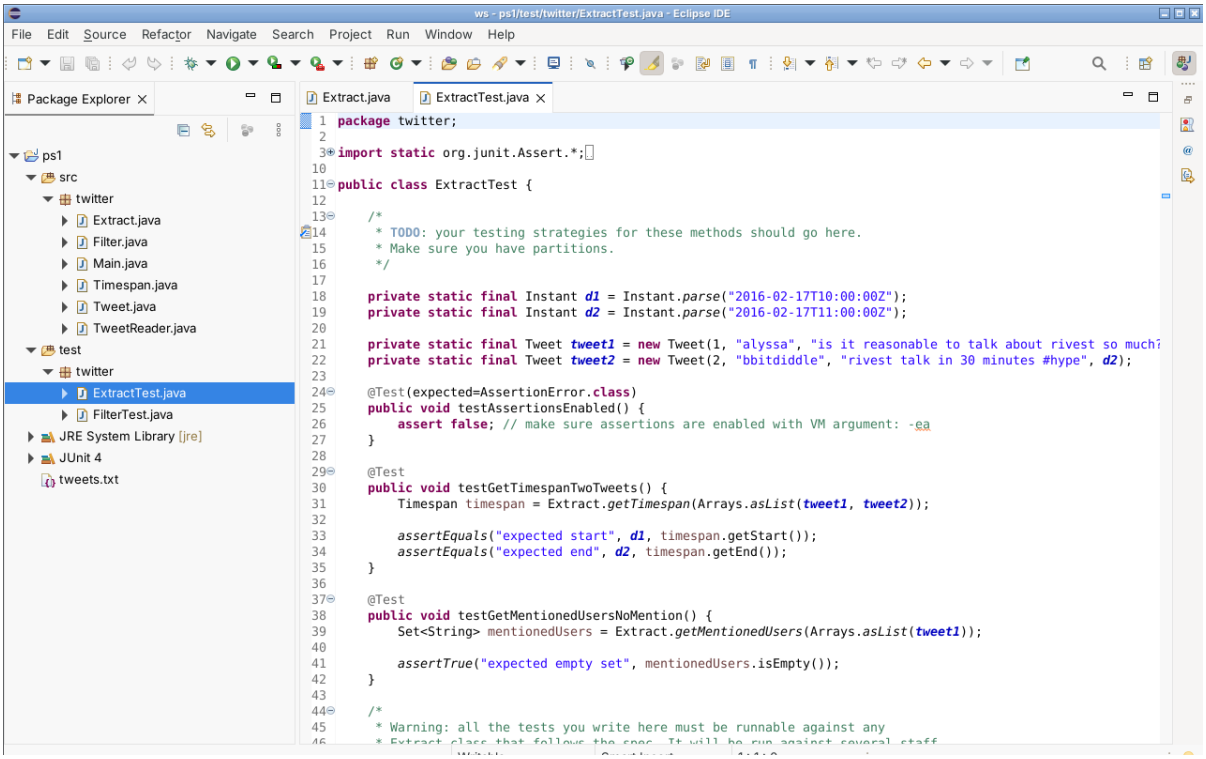


Figure 5: The starting ExtractTest.java

Right-click `ExtractTest.java` and choose **Run As > JUnit Test** (or press Alt+Shift+X and then T). The bar is red with 2 errors. The Failure Trace says "java.lang.RuntimeException: not implemented", which is correct because the methods have no code yet. This is the normal starting point of test-first programming.

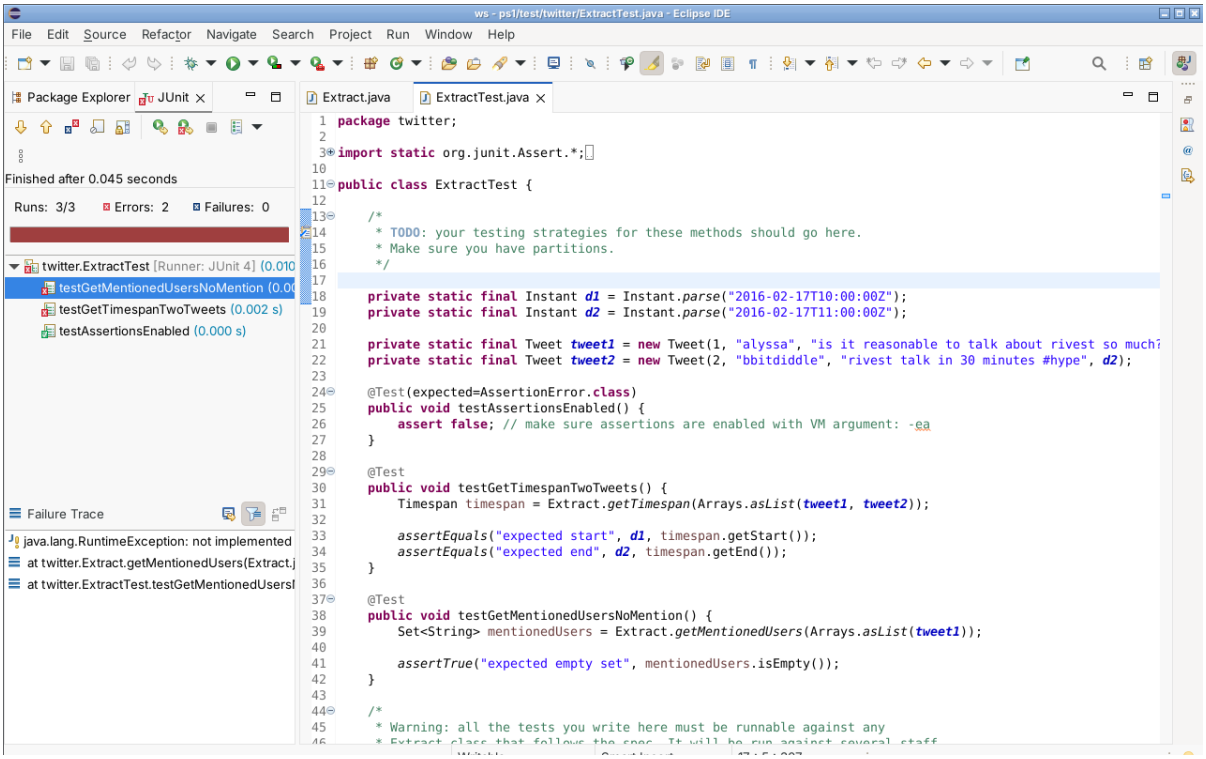


Figure 6: The given tests fail because Extract is not implemented

Note: `testAssertionsEnabled` must be green. It checks that Java assertions are switched on (the -ea option). Eclipse adds -ea to JUnit runs by default. If this test is red on your machine, go to **Window > Preferences > Java > JUnit** and tick the option that adds '-ea' to the VM arguments of new JUnit launch configurations, then delete the old run configuration and run again.

Step 4: Write the testing strategy

Replace the TODO comment at the top of `ExtractTest` with your strategy. Think about which inputs could make an implementation go wrong: one tweet or many, tweets not sorted by time, a mention at the start or end of the text, the same user written in different case, an email address, and so on. This is the strategy I used:

```
/*
 * Testing strategy for getTimespan(tweets):
 *   number of tweets: 1, > 1
 *   order of tweets in the list: in time order, not in time order
 *   timestamps: all the same, some different
 *   (empty list is not tested: the spec does not say what to return)
 *
 * Testing strategy for getMentionedUsers(tweets):
 *   number of tweets: 1, > 1
 *   mentions in a tweet: 0, 1, > 1
 *   same user mentioned: once, more than once with different case
 *   position of mention: start of text, middle, end
 *   "@" preceded by a username character (email address): yes, no
 *   mention followed by punctuation: yes, no
 *
 * The returned set may use any case for a username, so the tests
 * compare usernames after converting them to lowercase.
 */
```

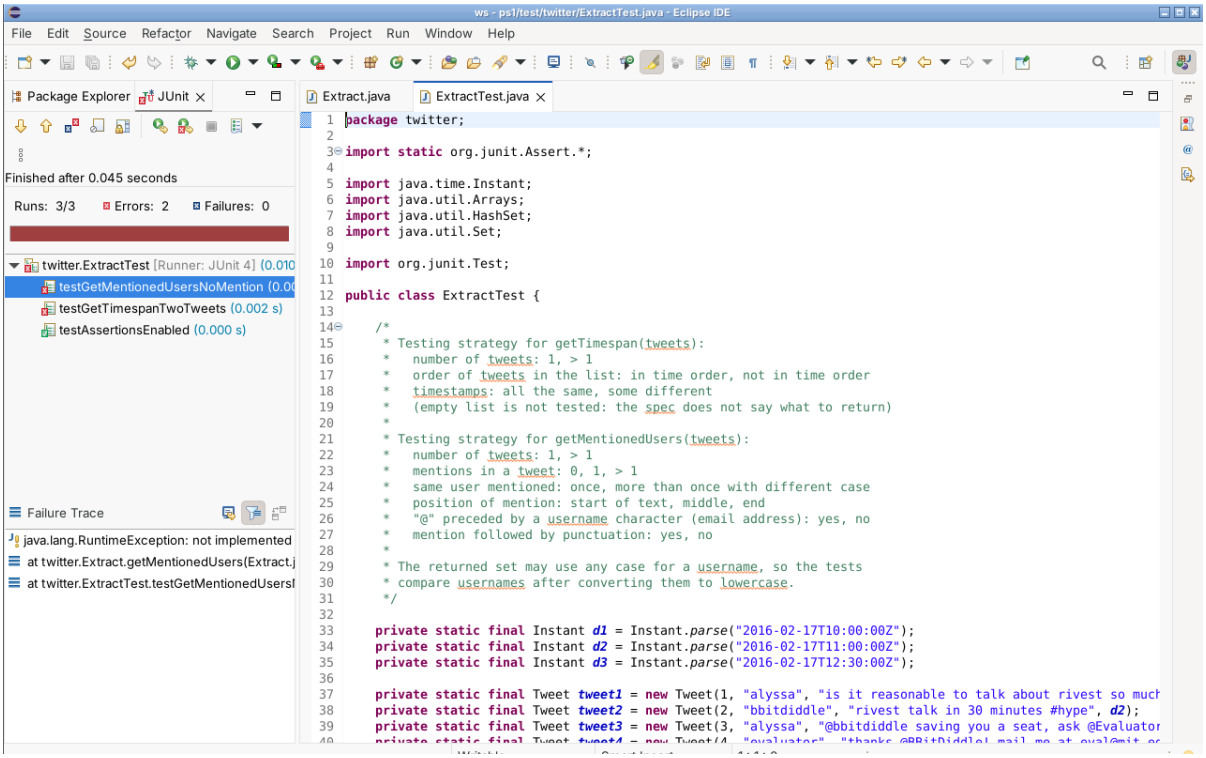


Figure 7: Testing strategy written at the top of ExtractTest

Step 5: Write the tests

First add some more tweets next to the two given ones, so that the cases from your strategy can be tested. In my solution the test data looks like this:

```
private static final Instant d1 = Instant.parse("2016-02-17T10:00:00Z");
private static final Instant d2 = Instant.parse("2016-02-17T11:00:00Z");
private static final Instant d3 = Instant.parse("2016-02-17T12:30:00Z");

private static final Tweet tweet1 =
        new Tweet(1, "alyssa", "is it reasonable to talk about rivest so much?", d1);
private static final Tweet tweet2 =
        new Tweet(2, "bbitdiddle", "rivest talk in 30 minutes #hype", d2);
private static final Tweet tweet3 =
        new Tweet(3, "alyssa", "@bbitdiddle saving you a seat, ask @Evaluator too", d3);
private static final Tweet tweet4 =
        new Tweet(4, "evaluator", "thanks @BBitDiddle! mail me at eval@mit.edu", d2);
private static final Tweet tweet5 = new Tweet(5, "cy_d_fect", "see you there @cy-d-
fect_2", d1);
```

Then write one test for each part of the strategy. Every test has a comment that says which parts it covers. A few of the tests are shown below; the rest follow the same pattern and are visible in the screenshot.

```
// covers: > 1 tweet, not in time order
@Test
public void testGetTimespanUnorderedTweets() {
    Timespan timespan = Extract.getTimespan(Arrays.asList(tweet3, tweet1, tweet2));

    assertEquals("expected start", d1, timespan.getStart());
    assertEquals("expected end", d3, timespan.getEnd());
}

// covers: "@" inside an email address is not a mention
@Test
public void testGetMentionedUsersEmailIsNotMention() {
    Set<String> mentionedUsers =
toLowerCase(Extract.getMentionedUsers(Arrays.asList(tweet4)));

    assertFalse("mit is not mentioned", mentionedUsers.contains("mit"));
    assertEquals("expected one user", new HashSet<>(Arrays.asList("bbitdiddle")),
mentionedUsers);
}

// covers: > 1 tweets, same user mentioned with different case
@Test
public void testGetMentionedUsersDifferentCase() {
    Set<String> mentionedUsers = Extract.getMentionedUsers(Arrays.asList(tweet3,
tweet4));

    assertEquals("expected each user once", 2, mentionedUsers.size());
    assertEquals("expected two users", new HashSet<>(Arrays.asList("bbitdiddle",
"evaluator")),
            toLowerCase(mentionedUsers));
}

// usernames are case-insensitive, so compare them in lowercase
private static Set<String> toLowerCase(Set<String> usernames) {
    Set<String> result = new HashSet<>();
    for (String username : usernames) {
        result.add(username.toLowerCase());
    }
    return result;
}
```

Look at how the tests deal with case. The spec allows "BBitDiddle" or "bbitdiddle" in the returned set, so the tests convert the result to lowercase with a small helper method before comparing. The helper is kept inside the test class and not in `Extract`, as the warning at the end of the file asks.

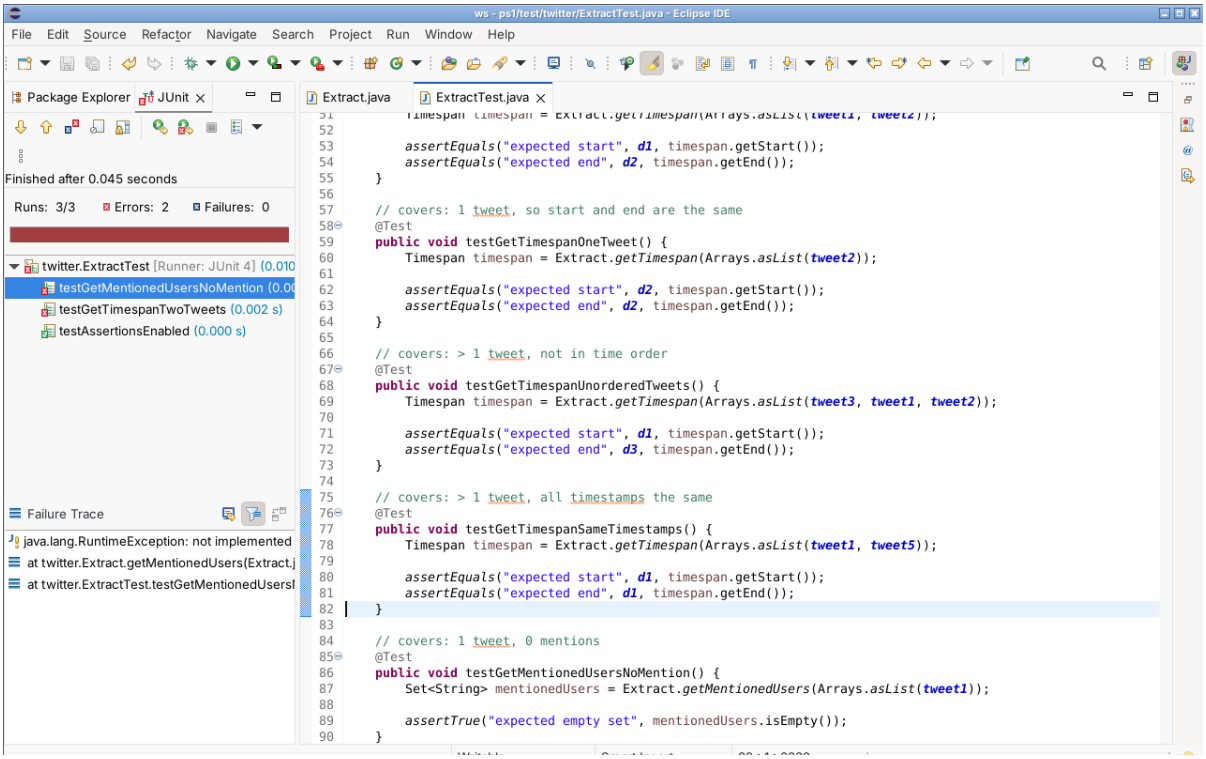


Figure 8: Some of the new tests for getTimespan

Step 6: Run the tests again

Run `ExtractTest` again. You should now see Runs: 10/10 with 9 errors. All of the new tests fail with "not implemented", and only `testAssertionsEnabled` passes. Take a screenshot of this for your report, because it shows that the tests were written before the code.

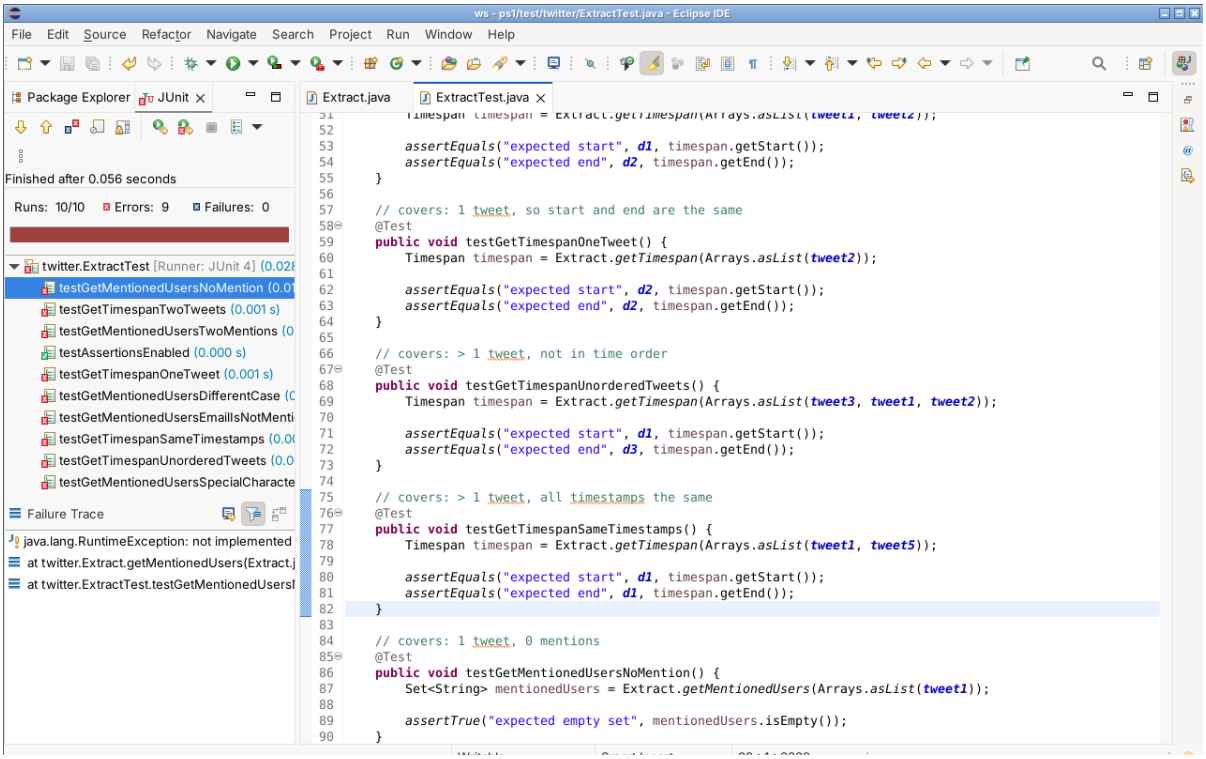


Figure 9: All new tests fail before the implementation

Step 7: Implement getTimespan

Remove the `throw` line and write the method. Start with the timestamp of the first tweet as both start and end, then go through all tweets and move the start back or the end forward when you find an earlier or later time.

```
public static Timespan getTimespan(List<Tweet> tweets) {
    if (tweets.isEmpty()) {
        // the spec does not say what to return here, so any timespan is fine
        return new Timespan(Instant.EPOCH, Instant.EPOCH);
    }
    Instant start = tweets.get(0).getTimestamp();
    Instant end = start;
    for (Tweet tweet : tweets) {
        Instant time = tweet.getTimestamp();
        if (time.isBefore(start)) {
            start = time;
        }
        if (time.isAfter(end)) {
            end = time;
        }
    }
    return new Timespan(start, end);
}
```

The empty-list case is not covered by the spec, so any answer is acceptable there. I return a zero-length timespan only so that the method does not crash. You will need `import java.time.Instant;` at the top of the file (Ctrl+Shift+O adds missing imports in Eclipse).

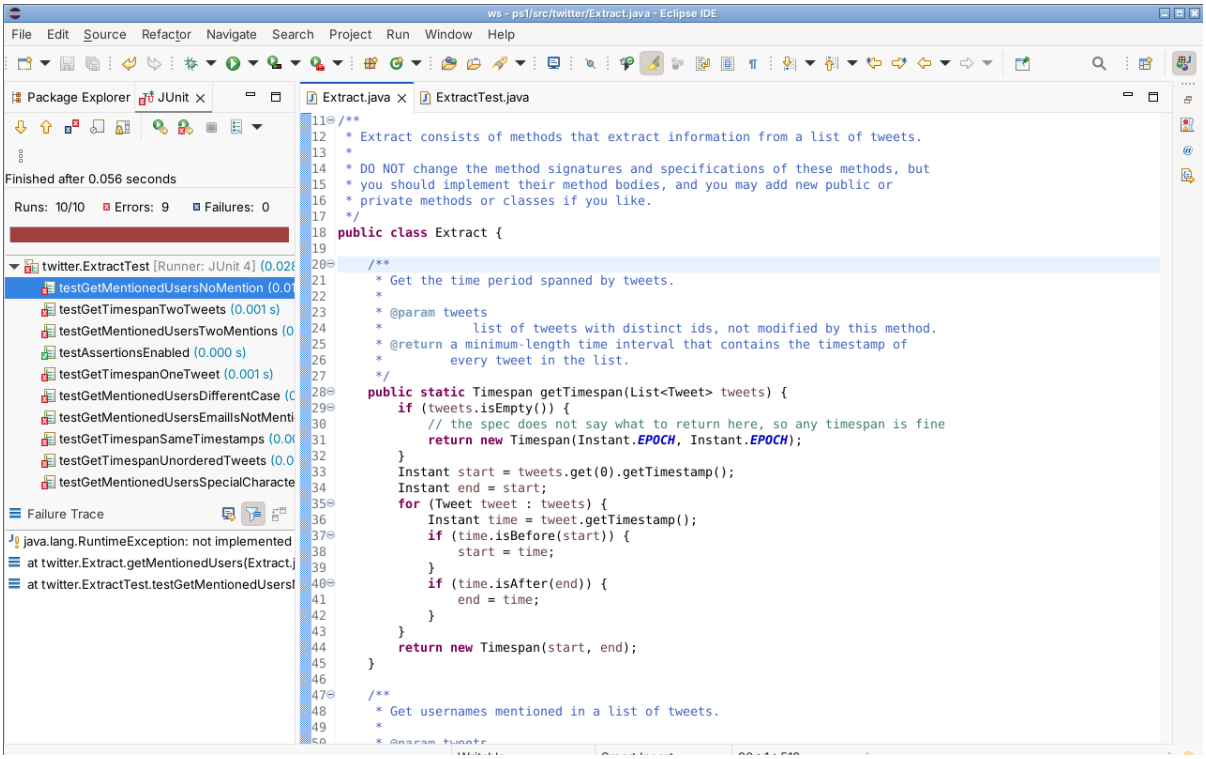


Figure 10: getTimespan implemented

Step 8: Implement getMentionedUsers

A regular expression makes this method short. The pattern `(?<![A-Za-z0-9_-])@([A-Za-z0-9_-]+)` means: an "@" that is not preceded by a username character, followed by one or more username characters. The `+` takes as many characters as it can, so the match can never be followed by another username character. Each match is put in a `HashSet` in lowercase, so the same user is stored only once.

```
public static Set<String> getMentionedUsers(List<Tweet> tweets) {
    Set<String> mentioned = new HashSet<>();
    for (Tweet tweet : tweets) {
        Matcher matcher = MENTION.matcher(tweet.getText());
        while (matcher.find()) {
            // usernames are case-insensitive, so keep one lowercase copy
            mentioned.add(matcher.group(1).toLowerCase(Locale.ROOT));
        }
    }
    return mentioned;
}

// "@" + username, not preceded by a username character; the greedy +
// makes sure it is not followed by one either
private static final Pattern MENTION = Pattern.compile("(?<![A-Za-z0-9_-])@([A-Za-z0-9_-]+)");
```

The imports needed are `java.util.HashSet`, `java.util.Locale`, `java.util.regex.Matcher` and `java.util.regex.Pattern`. If you are not comfortable

with regular expressions, you can also go through the text character by character. That also works, but it takes more code.

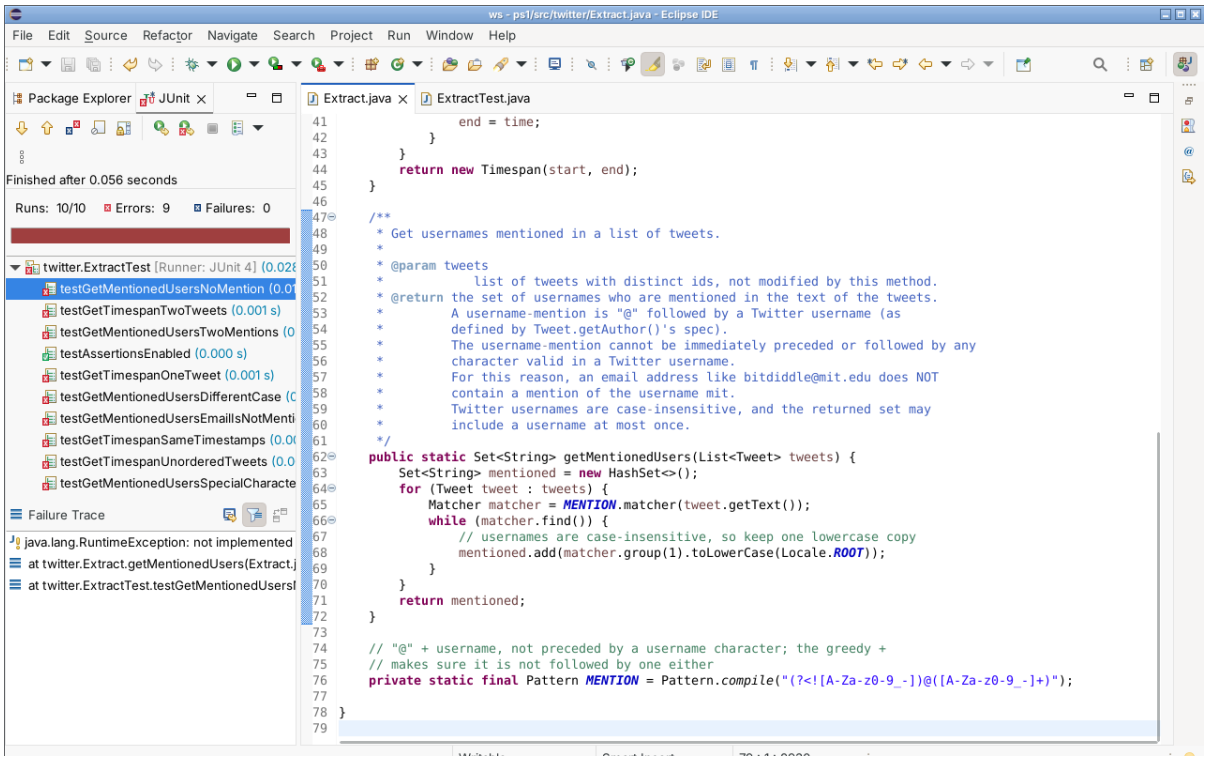


Figure 11: getMentionedUsers implemented

Step 9: Run the tests until they pass

Run `ExtractTest` once more (the Rerun Test button in the JUnit view does the same). All 10 tests should pass with a green bar. If any test fails, read the message in the Failure Trace and decide whether the code or the test is wrong. Do not change a test just to make it pass. Change it only if it checks something the spec does not promise.

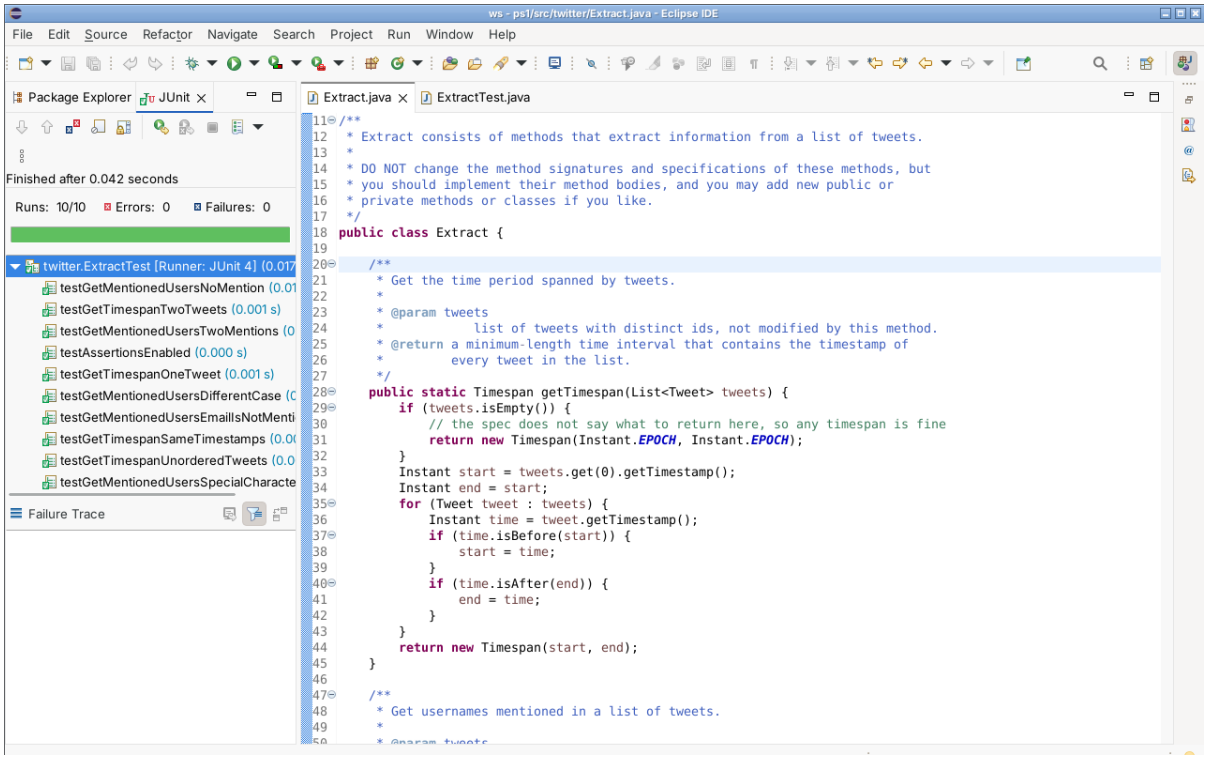


Figure 12: All ExtractTest tests pass

Step 10: Check that your tests can catch a bug

Tests are useful only if they fail when the code is wrong. As an exercise, remove `.toLowerCase(Locale.ROOT)` from `getMentionedUsers`, save, and run the tests again. Now "bbitdiddle" and "BBitDiddle" go into the set as two different strings.

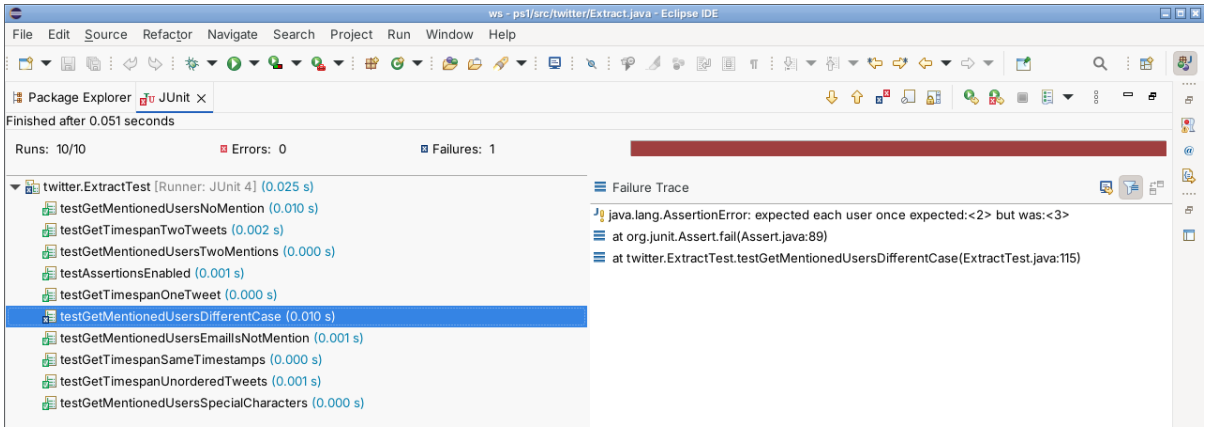


Figure 13: testGetMentionedUsersDifferentCase catches the missing lowercase conversion

`testGetMentionedUsersDifferentCase` fails with "expected each user once expected:<2> but was:<3>". This is the kind of mistake the hint in the manual is about. Put the `.toLowerCase(Locale.ROOT)` back and make sure the bar is green again before you go on.

Task 2: Filtering lists of tweets

Step 11: Read the specs of Filter

Open `Filter.java` and read the specs of `writtenBy`, `inTimespan` and `containing`.

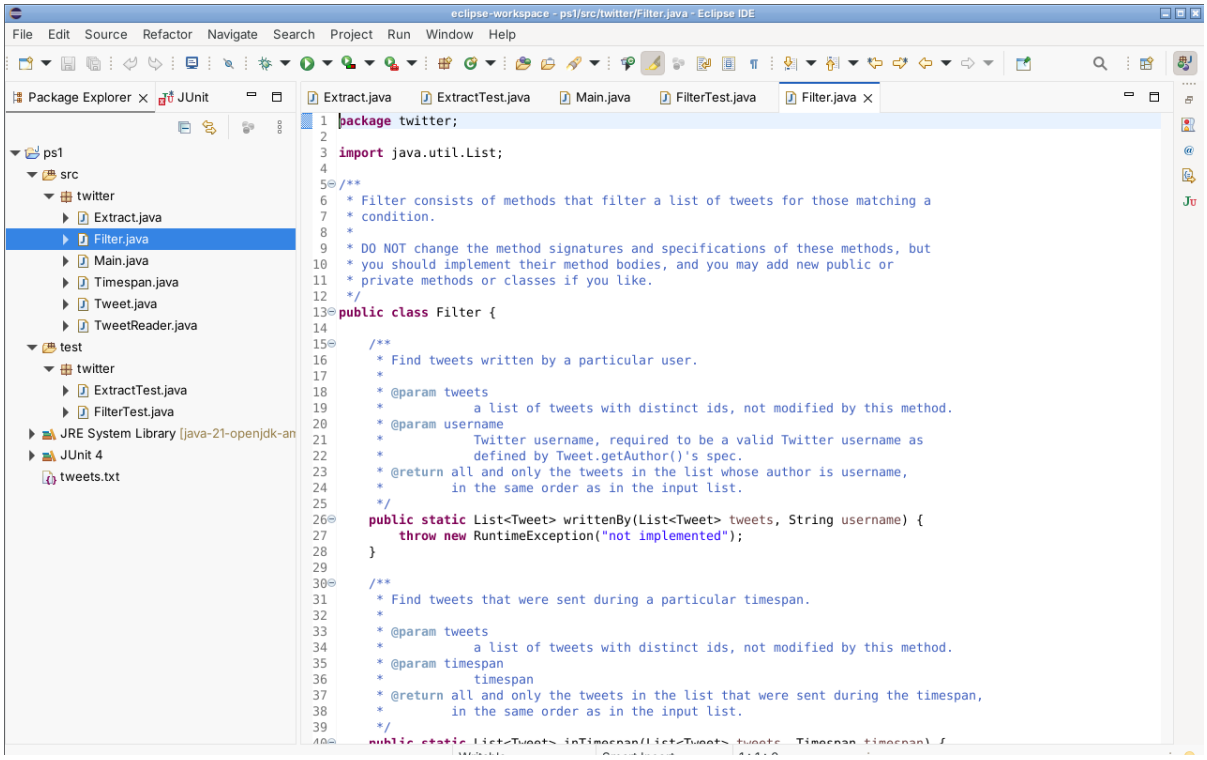


Figure 14: Specs in Filter.java

Points to notice:

- All three methods must return the tweets **in the same order as in the input list**. Your tests should check the order, not only which tweets come back.
- `writtenBy`: usernames are case-insensitive, so "ALYSSA" must also find the tweets of "alyssa".
- `inTimespan`: a Timespan includes its endpoints, so a tweet sent exactly at the start or the end time must be included.
- `containing`: the text is split into words at spaces, and a word must match completely (ignoring case). "Talking" does not contain the word "talk", but "talk" and "TALK" are the same word.

Step 12: Write the strategy and the tests for Filter

Do the same as in Task 1. Write the strategy at the top of `FilterTest.java`, add the test data, and then write the tests.

```
/*
 * Testing strategy for writtenBy(tweets, username):
 *   number of tweets: 0, 1, > 1
 *   number of tweets by username: 0, 1, > 1
 *   case of username: same as author, different from author
 *
 * Testing strategy for inTimespan(tweets, timespan):
 *   number of tweets: 0, > 1
 *   tweets inside timespan: none, some, all
 *   tweet exactly on start or end of timespan: yes, no
 *
 * Testing strategy for containing(tweets, words):
 *   number of words: 1, > 1
 *   matching tweets: none, some, all
 *   case of word: same as in tweet, different
 *   word appears only as part of a bigger word: yes, no
 *
 * Every result is also checked for keeping the input order.
 */
```

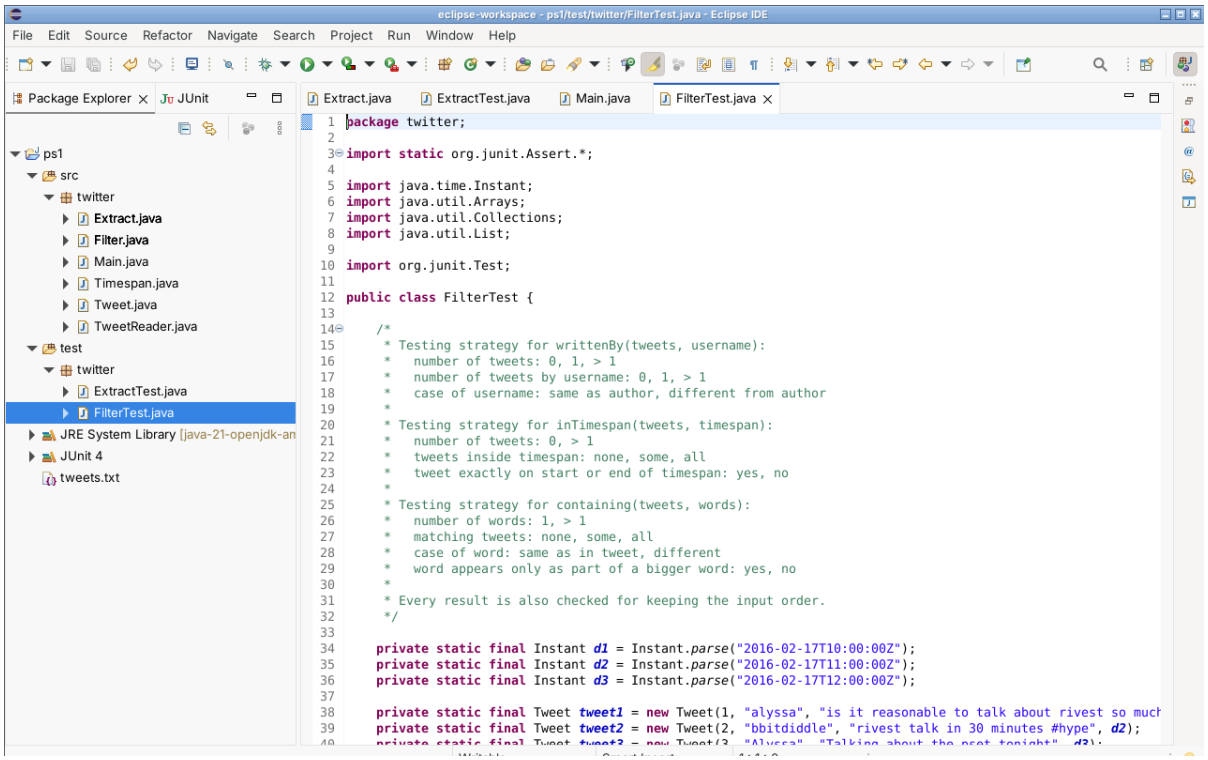


Figure 15: Testing strategy in FilterTest

I added one more tweet, written by "Alyssa" with a capital A, to test the case rule of `writtenBy` and the whole-word rule of `containing`:

```
private static final Instant d1 = Instant.parse("2016-02-17T10:00:00Z");
private static final Instant d2 = Instant.parse("2016-02-17T11:00:00Z");
private static final Instant d3 = Instant.parse("2016-02-17T12:00:00Z");

private static final Tweet tweet1 =
        new Tweet(1, "alyssa", "is it reasonable to talk about rivest so much?", d1);
private static final Tweet tweet2 =
        new Tweet(2, "bbitdiddle", "rivest talk in 30 minutes #hype", d2);
private static final Tweet tweet3 = new Tweet(3, "Alyssa", "Talking about the pset
tonight", d3);
```

Three of the tests are shown here. Comparing with `assertEquals` against a complete list checks the contents and the order in one line.

```
// covers: > 1 tweets by username, author written in a different case
@Test
public void testWrittenByDifferentCaseMultipleResults() {
    List<Tweet> writtenBy = Filter.writtenBy(Arrays.asList(tweet1, tweet2, tweet3),
"ALYSSA");

    assertEquals("expected two tweets in order", Arrays.asList(tweet1, tweet3),
writtenBy);
}

// covers: tweets exactly on start and end are included, one tweet outside
@Test
public void testInTimespanEndpointsIncluded() {
    List<Tweet> inTimespan = Filter.inTimespan(Arrays.asList(tweet1, tweet2, tweet3),
            new Timespan(d2, d3));

    assertEquals("expected tweets on the endpoints", Arrays.asList(tweet2, tweet3),
inTimespan);
}

// covers: different case, word only inside a bigger word ("Talking" is not "talk")
@Test
public void testContainingCaseAndWholeWords() {
    List<Tweet> containing = Filter.containing(Arrays.asList(tweet1, tweet2, tweet3),
            Arrays.asList("TALK"));

    assertEquals("expected tweets 1 and 2 only", Arrays.asList(tweet1, tweet2),
containing);
}
```

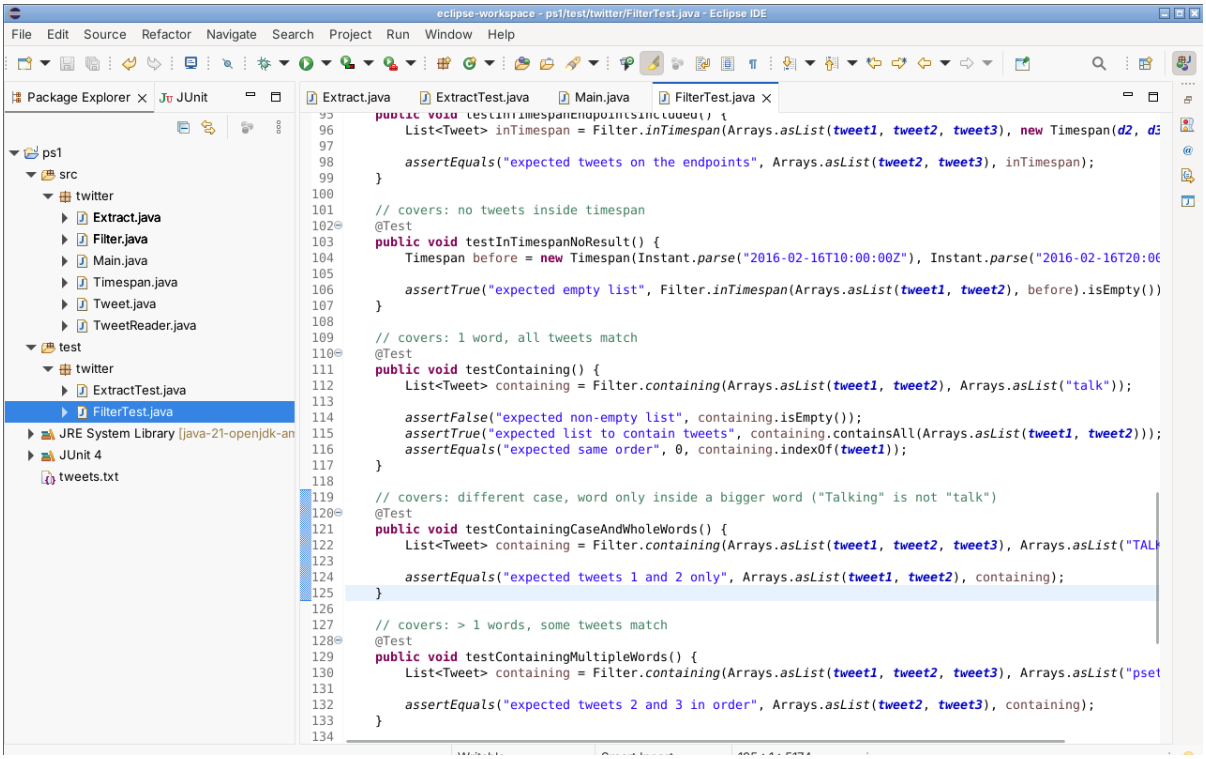


Figure 16: Some of the FilterTest tests

Step 13: Run FilterTest before implementing

Run `FilterTest`. As expected all tests except `testAssertionsEnabled` fail with "not implemented" (12 runs, 11 errors). Take a screenshot of this as well.

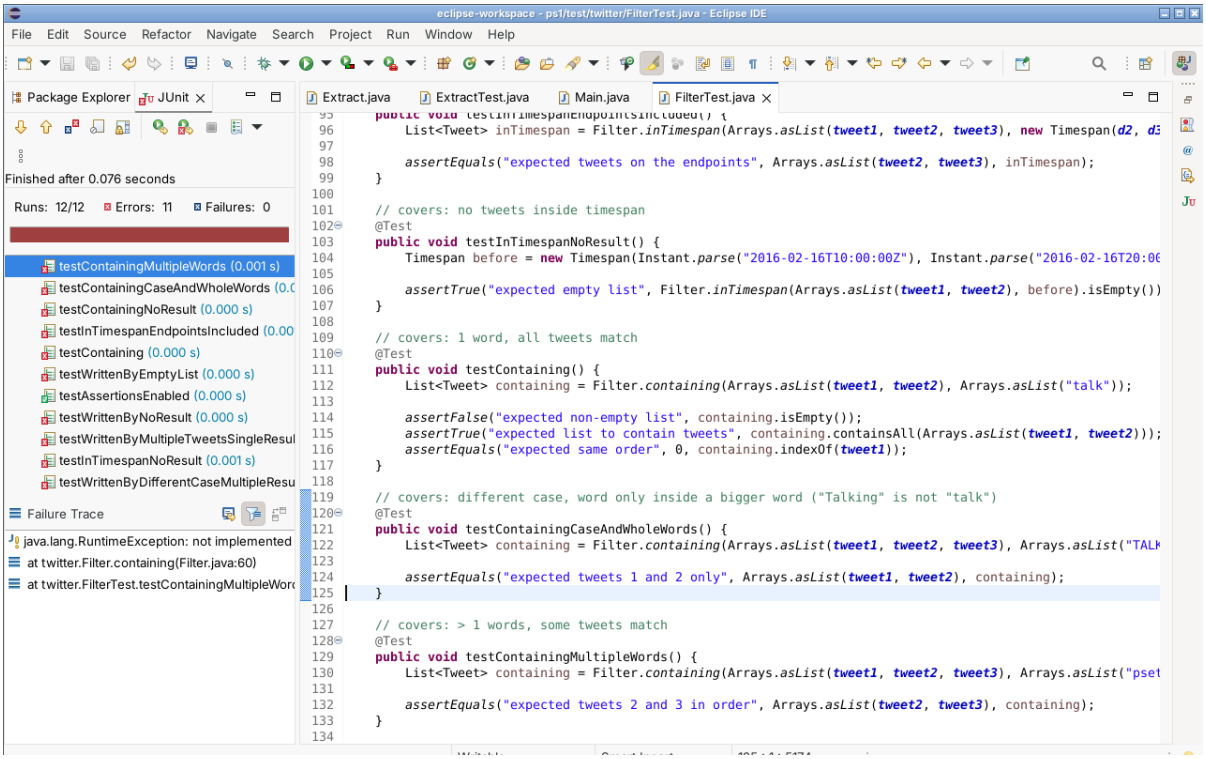


Figure 17: FilterTest fails before the implementation

Step 14: Implement the three methods

Each method makes a new `ArrayList`, goes through the input list in order and adds the tweets that match. Because the loop follows the input list, the order is kept automatically. Add `import java.util.ArrayList;` at the top.

```
public static List<Tweet> writtenBy(List<Tweet> tweets, String username) {
    List<Tweet> result = new ArrayList<>();
    for (Tweet tweet : tweets) {
        if (tweet.getAuthor().equalsIgnoreCase(username)) {
            result.add(tweet);
        }
    }
    return result;
}
```

```
public static List<Tweet> inTimespan(List<Tweet> tweets, Timespan timespan) {
    List<Tweet> result = new ArrayList<>();
    for (Tweet tweet : tweets) {
        // a Timespan includes its endpoints
        if (!tweet.getTimestamp().isBefore(timespan.getStart())
                && !tweet.getTimestamp().isAfter(timespan.getEnd())) {
            result.add(tweet);
        }
    }
    return result;
}
```

`inTimespan` uses "not before the start and not after the end" instead of `isAfter(start) && isBefore(end)`. The second form would leave out tweets that are exactly on the endpoints, and `testInTimespanEndpointsIncluded` would catch that.

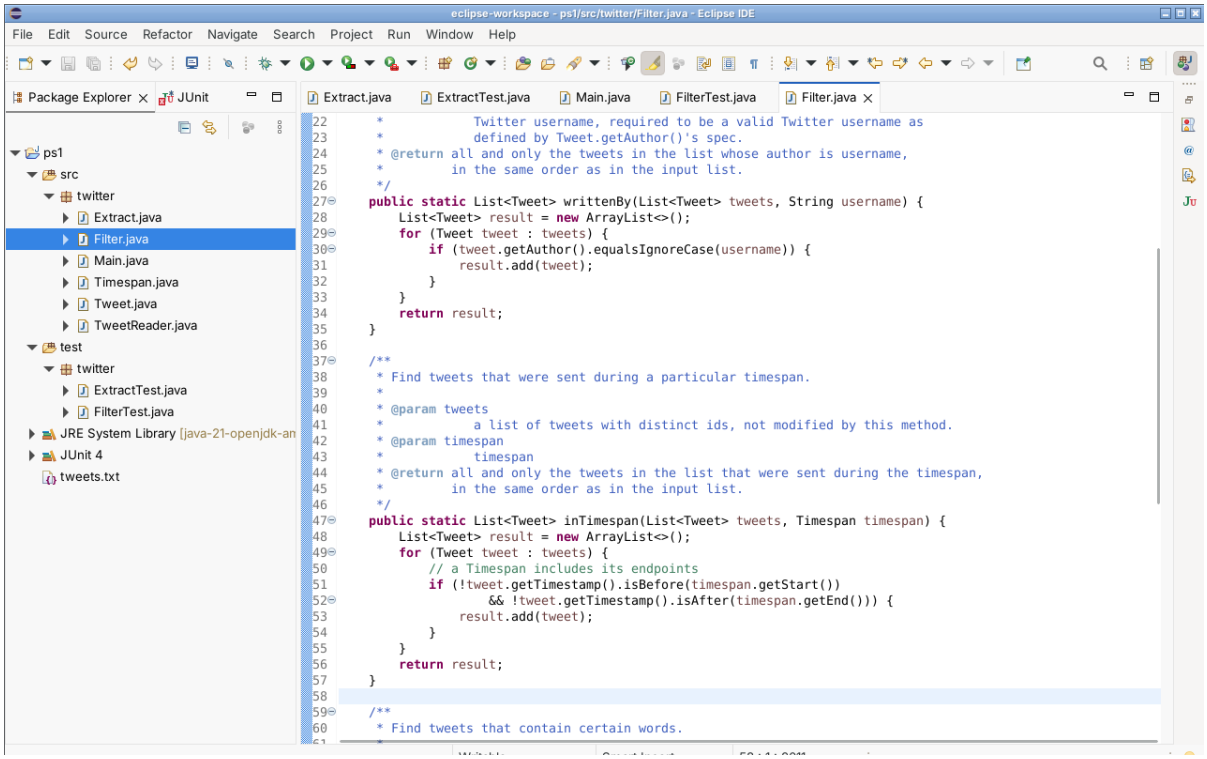


Figure 18: writtenBy and inTimespan implemented

For `containing`, a small private helper checks one tweet against the list of words:

```
public static List<Tweet> containing(List<Tweet> tweets, List<String> words) {
    List<Tweet> result = new ArrayList<>();
    for (Tweet tweet : tweets) {
        if (hasAnyWord(tweet.getText(), words)) {
            result.add(tweet);
        }
    }
    return result;
}

private static boolean hasAnyWord(String text, List<String> words) {
    for (String tweetWord : text.split(" ")) {
        for (String word : words) {
            if (!tweetWord.isEmpty() && tweetWord.equalsIgnoreCase(word)) {
                return true;
            }
        }
    }
    return false;
}
```

The check `!tweetWord.isEmpty()` is there because two spaces next to each other give an empty string after `split(" ")`, and the spec says words are nonempty.

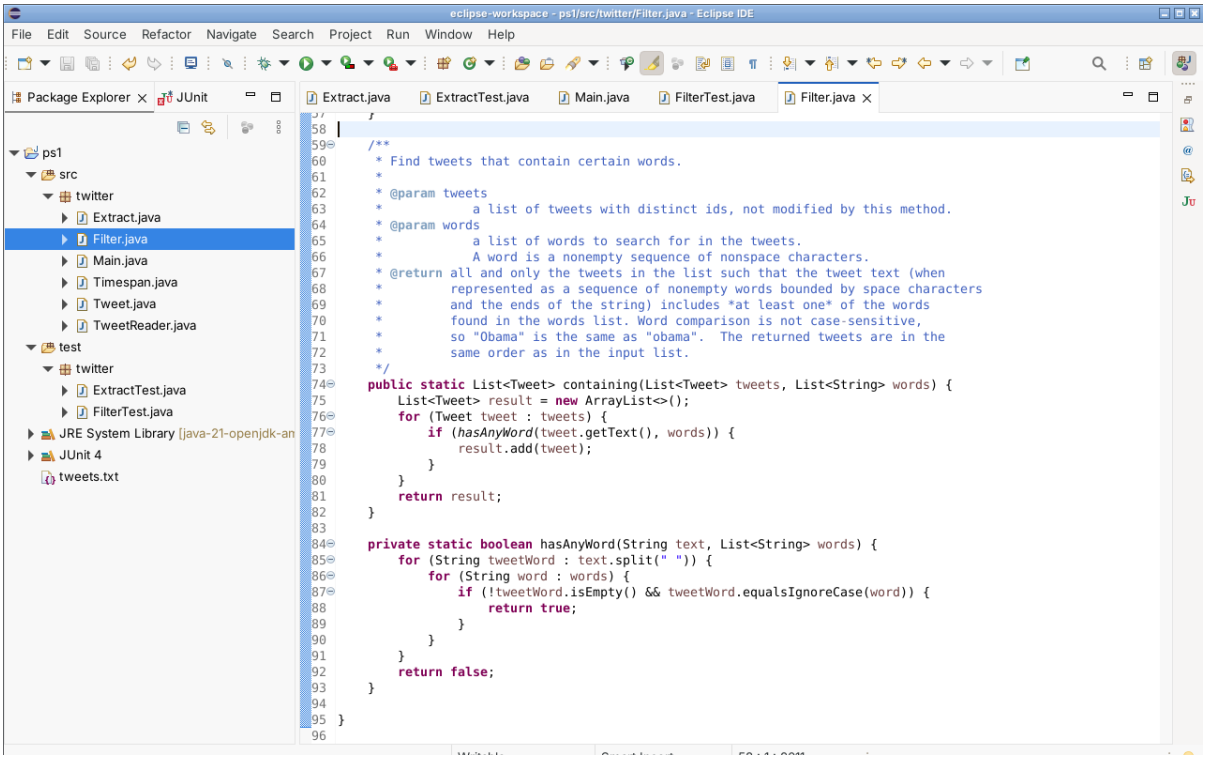


Figure 19: containing and its helper method

Step 15: Run FilterTest again

Run `FilterTest`. All 12 tests should pass.

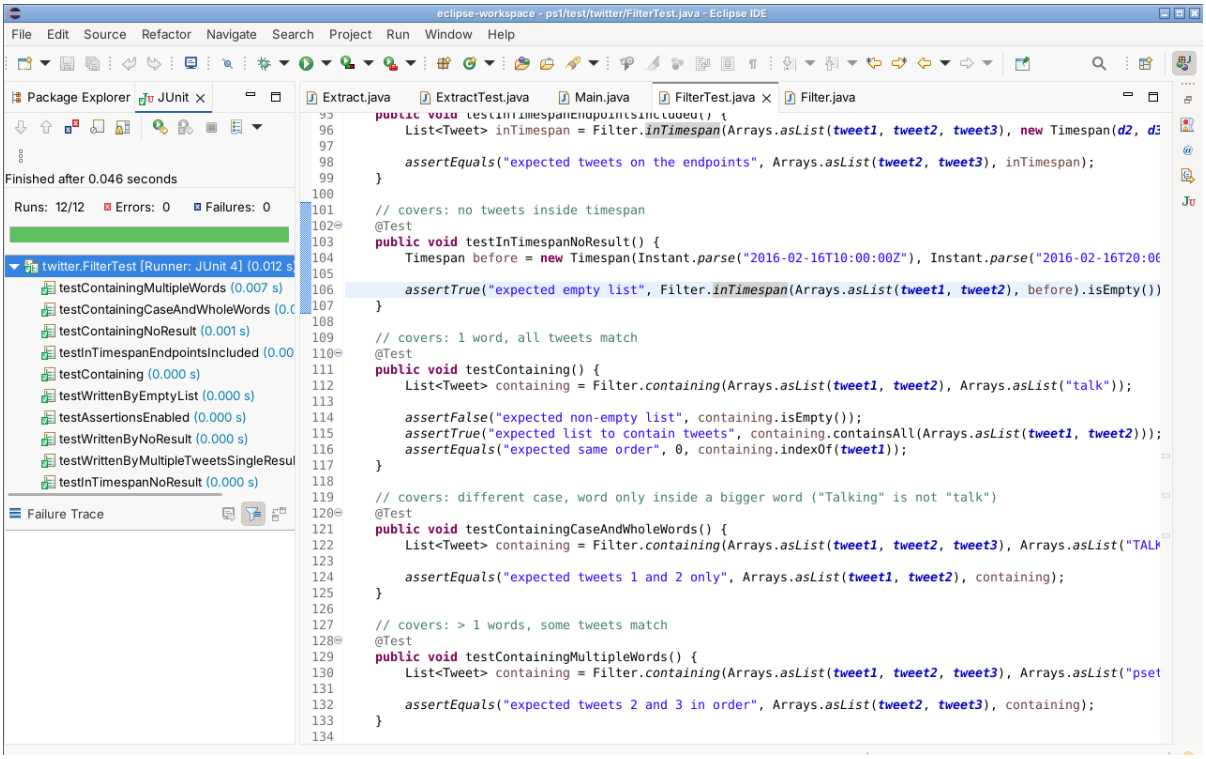


Figure 20: All FilterTest tests pass

Step 16: Run all the tests together

Right-click the **test** folder in the Package Explorer and choose **Run As > JUnit Test**. This runs both test classes in one go.

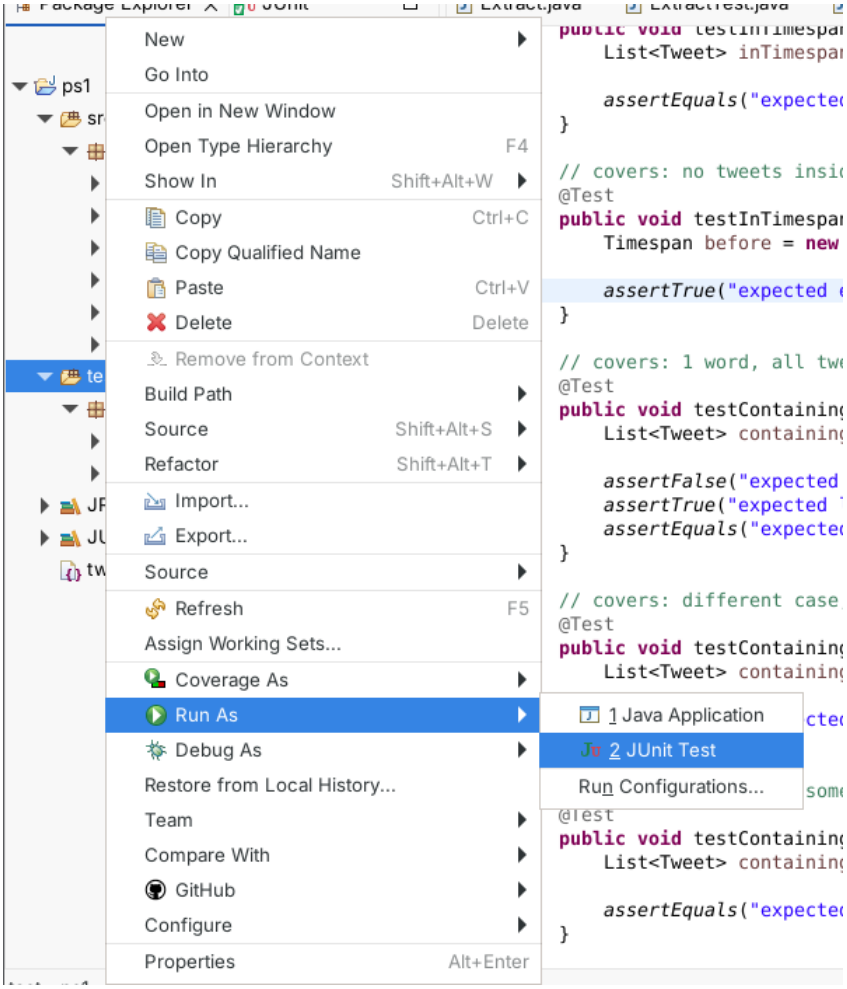


Figure 21: Running every test in the test folder

You should get Runs: 22/22 with no errors and no failures (10 tests in ExtractTest and 12 in FilterTest). Your numbers will be different if you wrote a different number of tests, but the bar must be green.

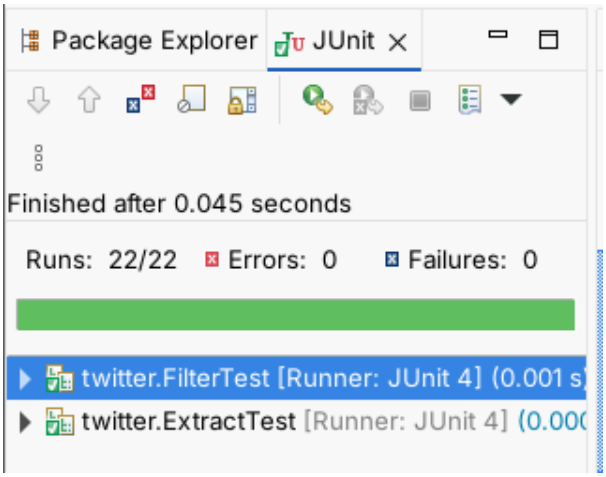


Figure 22: Both test classes pass

Step 17: Run Main

Finally, right-click `Main.java` and choose **Run As > Java Application**. Main reads the 8 sample tweets from `tweets.txt` and prints their timespan, the mentioned users, the tweets written by alyssa, and the tweets containing "rivest" or "pset". Main is not used in grading and you can change it as you like, for example to try other usernames or words.



Figure 23: Output of Main in the Console

Check the output against `tweets.txt`. "@Rivest" and "@BBitDiddle" are printed in lowercase, "cy@mit.edu" in tweet 5 is not counted as a mention, and tweet 7 is found for "pset" even though it is written "PSET".

What to submit

Fill the Solution part of the lab manual with the following, in this order:

1. Your testing strategy comments for ExtractTest and FilterTest (copy the text or take screenshots)
2. ExtractTest failing before the implementation (Step 6)
3. Your code of getTimespan and getMentionedUsers, and ExtractTest passing (Steps 7 to 9)
4. FilterTest failing before the implementation (Step 13)
5. Your code of writtenBy, inTimespan and containing, and FilterTest passing (Steps 14 and 15)
6. All tests passing together (Step 16) and the Console output of Main (Step 17)

Zip the ps1 project folder (or use File > Export > General > Archive File in Eclipse) and upload it on LMS with the Word file, one file per group. Screenshots taken from someone else's machine will be treated as unfair means, as written in the lab rubric.

Common problems

Problem	What to do
org.junit cannot be resolved	Right-click the project > Build Path > Add Libraries... > JUnit > JUnit 4.
No "JUnit Test" option in Run As	Right-click the test file (or the test folder), not a file from src.

Problem	What to do
testAssertionsEnabled is red	Assertions are off. Tick the -ea option under Window > Preferences > Java > JUnit, delete the old run configuration and run again.
Instant cannot be resolved	Add `import java.time.Instant;` or press Ctrl+Shift+O to organize imports.
A test fails only because of upper/lower case	Usernames may come back in any case. Compare them in lowercase in the test, as in Step 5.
A tweet on the start or end time is missing	Timespan includes its endpoints. Use `!isBefore(start) && !isAfter(end)`.
containing finds "talk" in "Talking"	Split the text into words and compare whole words with `equalsIgnoreCase`, instead of using `contains` on the whole text.
Main says tweets.txt not found (NoSuchFileException)	Run Main from inside Eclipse so the working directory is the project folder, and check that tweets.txt is in the project root next to src.
"has been compiled by a more recent version of the Java Runtime"	Project > Clean..., then run again. If it continues, set the compiler level to your installed Java under Project > Properties > Java Compiler.

If you are still stuck after trying these, ask the lab engineer during the lab instead of spending a long time on one error.